

Experiment 07: Input & Output Manual

Design a Form Using Express.js in Node.js

Student & Laboratory Credentials

- **Student Name:** Asmit Jogdand
- **Roll Number:** 25CE1051
- **Class / Division:** B.Tech Computer Engineering (Sem VI)
- **Academic Year:** 2026 – 2027
- **Course:** Skill Based Lab - Advanced Web Technology (SBL-AWT)
- **Institution:** Ramrao Adik Institute of Technology (RAIT), D Y Patil Deemed to be University, Nerul, Navi Mumbai
- **Problem Statement Domain:** HealthPulse — Digital Healthcare & Clinical Management Portal
- **Experiment Title:** Design a Form Using Express.js in Node.js

1. Experiment Aim

To design, implement, and validate an Express.js form processing application utilizing body-parser middleware, server-side data validation, dynamic state re-rendering with input preservation upon validation failure, dynamic consultation fee calculation, and receipt generation.

2. Input Specification

2.1 File System Input

File Name	Format	Role & Implementation Responsibility
server.js	Express.js (Node.js)	Defines Express application, body-parsing middleware (<code>express.urlencoded</code> , <code>express.json</code>), request logger, server-side validation engine, dynamic fee calculator, and HTML response builders.
package.json	JSON	Project dependencies (<code>express</code>) and run scripts (<code>npm start</code>).

2.2 Execution Command Input

```
# Navigate to the experiment directory
cd /home/ubuntu/SBL-EXPT/Asmit_25CE1051/Expt-07-ExpressJS-Form

# Step 1: Install dependencies
npm install

# Step 2: Start Express.js form server (Default Port: 4000)
node server.js

# Or with custom port:
PORT=4000 node server.js
```

2.3 Express Middleware Pipeline Input

```
// Body parsing for URL-encoded form submissions and JSON APIs
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// Custom request logger middleware
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});
```

2.4 Server-Side Validation Rules & Dynamic Pricing Matrix

FIELD NAME	SERVER-SIDE VALIDATION RULE / REGEX	REJECTION CONDITION / MESSAGE
patientName	name && name.trim().length >= 3	"Full Name required (min 3 chars)"
contactNumber	/^[6-9]\d{9}\$/.test(phone)	"Valid 10-digit mobile required"
emailAddress	/^[^\s@]+@[^\s@]+\.[^\s@]+\$/.test(email)	"Valid email format required"
specialist	specialist && specialist !== ""	"Please select a consulting doctor"
visitType	Telehealth / OPD / Home Visit	Determines dynamic consultation fee

Dynamic Fee Calculation Rules:

- Telehealth Consultation : ₹500
- Hospital OPD Visit : ₹800
- Specialist Home Visit : ₹1,500

2.5 Sample Test Input Payloads

Test Vector A: Invalid Incomplete Form Submission (Validation Failure)

```
curl -X POST http://localhost:4000/consultation \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "patientName=As&contactNumber=12345&emailAddress=asmit&specialist=&visitType=Telehealth"
```

Test Vector B: Complete Valid Consultation Booking (Validation Success)

```
curl -X POST http://localhost:4000/consultation \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "patientName=AsmitJogdand&contactNumber=9876543210&emailAddress=asmit%40healthpulse.org&specialist=Dr.+Amarsinh+V.+Vidhate+(Cardiology)&visitType=Telehealth&symptoms=Routine+EHR+Cardiac+Review"
```

3. Output Specification

3.1 Terminal Execution & Request Logging Output

```
$ PORT=4000 node server.js
[HealthPulse] Express.js Form Server running at http://localhost:4000
[HealthPulse] Author: Asmit Jogdand (25CE1051)
[2026-10-01T09:31:26.477Z] HEAD /
[2026-10-01T09:31:26.514Z] POST /consultation -> Validation Failed: 3 constraint errors detected.
[2026-10-01T09:31:26.532Z] POST /consultation -> Validation Passed: Consultation booked: HP-CONS-882194
[2026-10-01T09:31:28.120Z] GET /api/appointments -> Returned 1 booked consultation record.
```

3.2 HTTP 400 Bad Request Output (Validation Failure with Input Preservation)

When an invalid form is submitted, the server responds with `HTTP 400 Bad Request`, highlights the invalid fields in red, preserves the valid values, and prints an itemized error summary banner:

```
HTTP/1.1 400 Bad Request
Content-Type: text/html; charset=utf-8

+-----+
|  ▢ PLEASE CORRECT THE FOLLOWING VALIDATION ERRORS BEFORE PROCEEDING:  |
+-----+
|  • Patient full name must contain at least 3 characters.                |
|  • Contact number must be a valid 10-digit Indian cellular number (starting 6-9). |
|  • Please provide a syntactically valid email address (e.g. name@domain.com). |
|  • You must select a consulting specialist physician from the medical directory. |
+-----+
| Form Inputs (Preserved in State):                                       |
| Patient Name:  [ As ] (Border: Red - Invalid)                          |
| Contact Phone: [ 12345 ] (Border: Red - Invalid)                      |
| Email Address: [ asmit ] (Border: Red - Invalid)                     |
| Consulting Dr: [ -- Select Physician -- v ] (Border: Red - Invalid)   |
+-----+
```

3.3 HTTP 200 OK Consultation Receipt Output (Validation Success)

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

+-----+
|  ▢ HEALTHPULSE MEDICAL CLINIC – OFFICIAL CONSULTATION RECEIPT  |
+-----+
| Consultation Booking ID:  HP-CONS-882194                          |
| Issued Date & Time:       2026-10-01 10:45:00 IST                |
| Clinical Portal Status:   CONFIRMED & SCHEDULED                  |
|                                                                     |
| Patient Summary:                                                 |
| • Patient Legal Name:     Asmit Jogdand                         |
| • Registered Contact:     +91-9876543210                       |
| • Confirmation Email:     asmit@healthpulse.org                 |
|                                                                     |
| Medical Appointment Details:                                     |
| • Consulting Physician:   Dr. Amarsinh V. Vidhate (Department of Cardiology) |
| • Consultation Mode:      Telehealth Virtual Consultation       |
| • Chief Medical Complaint: Routine EHR Cardiac Review          |
| • Applicable Fee:         ₹500.00 (Inclusive of Clinical Telehealth Access) |
|                                                                     |
| [ 📄 Download Consultation Receipt (PDF) ] [ ▢ Add to Google Calendar ] |
+-----+
```

3.4 JSON API Export Output (`GET /api/appointments`)

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

[
  {
    "id": "HP-CONS-882194",
    "patientName": "Asmit Jogdand",
    "contactNumber": "9876543210",
    "emailAddress": "asmit@healthpulse.org",
    "specialist": "Dr. Amarsinh V. Vidhate (Cardiology)",
    "visitType": "Telehealth",
    "fee": 500,
    "symptoms": "Routine EHR Cardiac Review",
    "bookedAt": "2026-10-01T09:31:26.532Z"
  }
]
```

3.5 Test Case Execution Results

Test Scenario	Input Data Submitted	Express Middleware & Controller Logic	Observed Output Result	Status
Initial Form Load	GET /	Serves initial clean HTML form	HTTP 200; Clean booking form with default ₹800 OPD selected	PASS
Blank Form Submit	All fields blank	Triggers 4 validation error rules	HTTP 400; Form re-rendered with 4 itemized error bullet points	PASS
Invalid Cellular	phone="12345"	Regex <code>/^[6-9]\d{9}\$/</code> evaluates false	HTTP 400; Contact field highlighted in red border	PASS
Telehealth Rate	visitType="Telehealth"	Evaluates fee mapping logic	Receipt reflects dynamically calculated fee of ₹500	PASS
Home Visit Rate	visitType="Home Visit"	Evaluates fee mapping logic	Receipt reflects dynamically calculated fee of ₹1,500	PASS
Valid Submission	Asmit Jogdand complete	Validation passes; pushes to appointments	HTTP 200; Generated official receipt HP-CONS-882194	PASS
JSON API Verification	GET /api/appointments	Serializes memory store to JSON	HTTP 200; Returns array containing newly booked consultation	PASS

4. Conclusion & Verification

Experiment 07 demonstrated comprehensive Express.js server-side form handling. Middleware correctly decoded URL-encoded and JSON payloads, rigorous validation caught erroneous inputs while preserving user data, and dynamic business logic calculated appropriate medical fees and generated confirmation receipts.